



Data Structures and Algorithms

Title: Algorithmic and Mathematical Functions

Lesson No: 2

I. INTRODUCTION

A good programmer is someone who always looks both ways before crossing a one-way street.
~Doug Linder

Learning how to program requires not just knowing the syntax and semantics but, also you must be able to know how everything works inside your code. Working as a programmer in IT industry would not be an easy job. You must have that fun and passion in programming that will drive you to love your work as you will always be in front of your computer doing some coding at the same time making your analytical, logical and critical thinking mind works all the time.

One way to make programming fun and interesting is when you try to apply the algorithms of your favorite game in creating your own game design. Applying those algorithmic and mathematical notations as part of how you design and structure your data in building those code blocks.

This module will show and help you to understand how mathematics is related to computer as it simulates some mathematical notations that turns out to be the control structures that we are using in creating programs. This will form and improve your analytical and logical thinking by applying it to your program to get a good result and to become a good programmer.

II. LEARNING OBJECTIVES

After studying this module, you should be able to:

- Understand some mathematical notations being used as algorithms and control constructs in programming.
- Solve mathematical functions and use it in programming.
- Apply mathematical functions in solving programming problems.
- Create and design programs that uses mathematical notations in building new algorithms and programming constructs.



III. TOPICS AND KEY CONCEPTS

A. Becoming a Good Programmer

There is no definition of a good programmer, as long as you can build codes that will solve a specific given problem: starting from the basic, writing your code, understanding your program flow, improving your skills in writing the codes for better understanding are good enough to be called as a good programmer. But, we do not just settle on becoming good, as time goes by we need to be better programmer who can develop new techniques, new constructs and new algorithms. Or at far, be the best programmer who can build new programming languages, introduce new platform and develop IT solutions to a certain company or industry.

In addition, you can also read the online article written by Ashish Arya on 10 effective ways to become a good programmer summarized as:

1. Work on Basics
2. Start putting question tags (how, what) with every set of code you write
3. You learn more by helping others
4. Write simple, understandable but logical code
5. Spend more time in analyzing the problem, you'll need less time to fix it
6. Be the first to analyze and review your code
7. Don't dismay yourself by looking at changing technology world
8. Work-arounds don't work for longer time
9. Read documentation
10. You can learn from others code as well

B. Algorithmic Notations

A notation is a series or system of written symbols used to represent numbers, amounts, or elements in something such as music or mathematics. A mathematical notation is a writing system used for recording concepts in mathematics. The notation uses symbols or symbolic expressions that are intended to have a precise semantic meaning. In the history of mathematics, these symbols have denoted numbers, shapes, patterns, and change.

An algorithm is a sequence of computational steps that transform the input into the output. An algorithm is a sequence of operations performed on data that have to be organized in data structures. An algorithm is a set of instructions designed to perform a specific task. This can be a simple process, such as multiplying two numbers, or a complex operation, such as playing a compressed video file.

The algorithmic notation is best described with the aid of examples. Consider the following algorithm which determines the largest algebraic element of a vector:



Algorithm GREATEST. This algorithm finds the largest algebraic element of vector A which contains N elements and places the result in MAX. I is used to subscript A.

1. [Is the vector empty?]
 If $N < 1$
 then Write ('EMPTY VECTOR')
 Exit
2. [Initialize]
 $MAX \leftarrow A[1]$ (We assume initially that $A[1]$ is the greatest element)
 $I \leftarrow 2$
3. [Examine all elements of vector]
 Repeat through step 5 while $I \leq N$
4. [Change MAX if it is smaller than the next element]
 If $MAX < A[I]$
 then $MAX \leftarrow A[I]$
5. [Prepare to examine next element in vector]
 $I \leftarrow I + 1$
6. [Finished]
 Exit

Figure 1. Algorithmic Notation Sample

The execution of an algorithm begins at step 1 and continues from there in sequential order unless the result of a condition tested or an unconditional transfer ('Go to', 'Exitloop', or 'Exit') specifies otherwise. In the sample algorithm, step 1 is executed first. If vector A is empty, the algorithm terminates; otherwise, step 2 is performed. In this step MAX is initialized to the value of $A[1]$ and the subscript variable, I, is assigned the value 2. Step 3 leads to the termination of this algorithm if we have already tested the last element of A; otherwise, step 4 is performed. In this step, the value of MAX is compared with the value of the next element of the vector. If MAX is less than the next element, then MAX is assigned this new value. If the test fails, no reassignment takes place. The completion of step 4 is followed by step 5, where the subscript value is incremented; control then returns to the testing step, step 3.

FORMAT CONVENTIONS

The following subsections summarize the basic format conventions used in the formulation of algorithms.

Name of Algorithm

Every algorithm is given an identifying name (GREATEST in the sample algorithm) written in capital letters.



Introductory Comment

The algorithm name is followed by a brief description of the tasks the algorithm performs and any assumptions that have been made. The description gives the names and types of the variables used in the algorithm.

Steps

The actual algorithm is made up of a sequence of numbered steps, each beginning with a phrase enclosed in square brackets which gives an abbreviated description of that step. Following this phrase is an ordered sequence of statements which describe actions to be executed or tasks to be performed. The statements in each step are executed in a left-to-right order.

Comments

An algorithm step may terminate with a comment enclosed in round parentheses intended to help the reader better understand that step. Comments specify no action and are included only for clarity.

Go To and Exitloop Statements

These two statements are rather extraordinary and are seldom used. The go to statement causes unconditional transfer of control to the step referenced. Thus, "Go to step N" will cause transfer of control to step N regardless of whether the statements of a loop or of a then or else clause have been completed. The exitloop statement is similar to the go to statement, but the range to which it can transfer control is limited. As the name implies, exitloop causes an immediate, unconditional exit from a loop. Note that an exitloop statement applies to only one level; thus, if one loop is nested within another, an exitloop from the inner loop will transfer control back to the outer loop. In general, go to and exitloop statements should be avoided since they lead to unstructured algorithms and, ultimately, to bad programming practice.

Exit Statement

The exit statement is used to terminate an algorithm. It is usually the last step:

```
      :  
      :  
7. [Finished]  
   Exit
```

However, there are often situations in which one may wish termination in midstream:

```
      :  
      :  
[Check to see if processing can continue]  
  If condition  
  then Exit  
      :
```

In any case, "Exit" causes immediate termination of an algorithm.



A variable is an entity that possesses a value, and its name is chosen to reflect the meaning of the value it holds (in the sample algorithm, MAX holds the largest algebraic element of a vector). For our purposes, a variable name always begins with a letter followed by characters which may be chosen from a set of possible characters including letters, numeric digits, and some special characters. Blanks are not permitted within a name, and all letters are capitalized. The following are examples of valid variable names:

C. Properties and Complexities of Algorithms

When writing an algorithm, there are some established mechanical procedure and properties to consider:

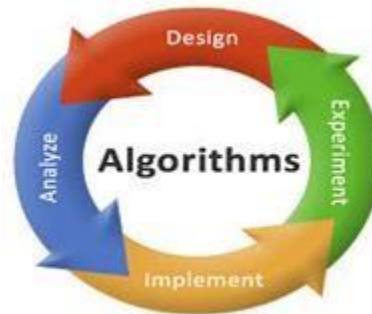


Figure 2. Procedures in writing algorithm

- Finiteness. An algorithm must always terminate after a finite number of steps.
- Definiteness. Each step of an algorithm must be precisely defined; the actions to be carried out must be rigorously and unambiguously specified for each case.
- Input. An algorithm has zero or more inputs, i.e, quantities which are given to it initially before the algorithm begins.
- Output. An algorithm has one or more outputs i.e, quantities which have a specified relation to the inputs.
- Effectiveness. An algorithm is also generally expected to be effective. This means that all of the operations to be performed in the algorithm must be sufficiently basic that they can in principle be done exactly and in a finite length of time.



I. Complexity of Algorithm:

It is very convenient to classify algorithms based on the relative amount of time or relative amount of space they require and specify the growth of time /space requirements as a function of the input size. Thus, we have the notions of:

1. **Time Complexity:** Running time of the program as a function of the size of input
2. **Space Complexity:** Amount of computer memory required during the program execution, as a function of the input size.

II. Asymptotic Notations

Asymptotic Notations are the expressions that are used to represent the complexity of an algorithm.

Best Case: In which we analyze the performance of an algorithm for the input, for which the algorithm takes less time or space.

Worst Case: In which we analyze the performance of an algorithm for the input, for which the algorithm takes long time or space.

Average Case: In which we analyze the performance of an algorithm for the input, for which the algorithm takes time or space that lies between best and worst case.

III. Types of Data Structure Asymptotic Notation

1. **Big-O Notation (O)** – Big O notation specifically describes worst case scenario.
2. **Omega Notation (Ω)** – Omega(Ω) notation specifically describes best case scenario.
3. **Theta Notation (Θ)** – This notation represents the average complexity of an algorithm.

Big-O Notation (O)

Big O notation specifically describes worst case scenario. It represents the upper bound running time complexity of an algorithm. Let's take few examples to understand how we represent the time and space complexity using Big O notation.

$O(1)$

Big O notation $O(1)$ represents the complexity of an algorithm that always execute in same time or space regardless of the input data.



The following step will always execute in same time(or space) regardless of the size of input data.

Accessing array index(int num = arr[5])

$O(n)$

Big O notation $O(n)$ represents the complexity of an algorithm, whose performance will grow linearly (in direct proportion) to the size of the input data.

The execution time will depend on the size of array. When the size of the array increases, the execution time will also increase in the same proportion (linearly)

Traversing an array

$O(n^2)$

Big O notation $O(n^2)$ represents the complexity of an algorithm, whose performance is directly proportional to the square of the size of the input data.

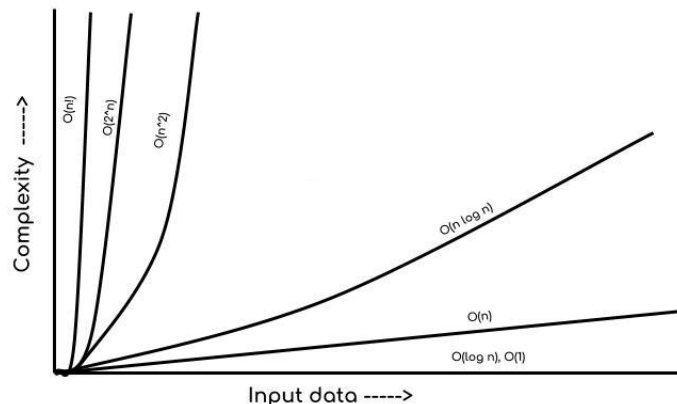
Traversing a 2D array

Other examples: Bubble sort, insertion sort and selection sort algorithms (we will discuss these algorithms later in another module)

Similarly, there are other Big O notations such as:

logarithmic growth $O(\log n)$, log-linear growth $O(n \log n)$, exponential growth $O(2^n)$ and factorial growth $O(n!)$.

If I have to draw a diagram to compare the performance of algorithms denoted by these notations, then I would draw it like this:

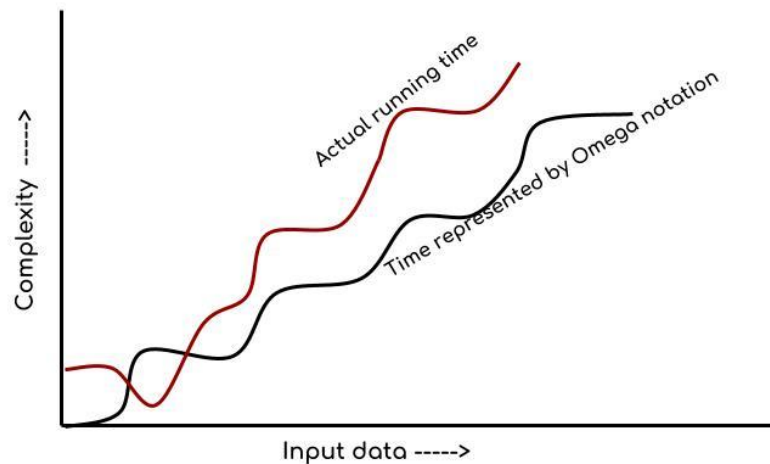


$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$



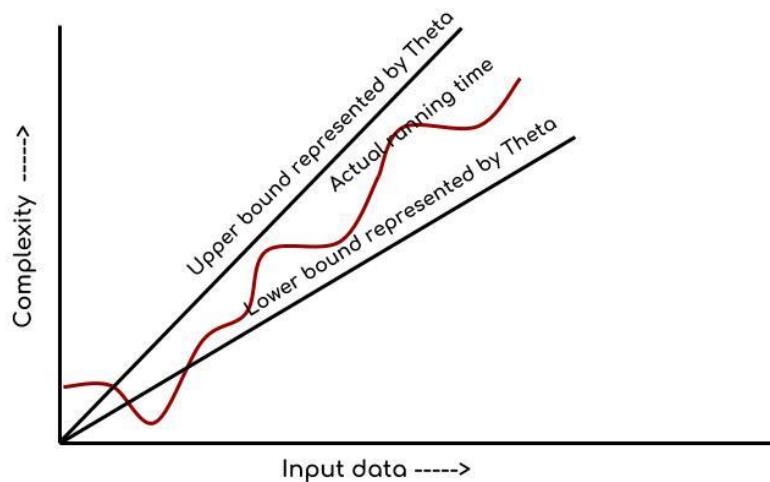
Omega Notation (Ω)

Omega notation specifically describes best case scenario. It represents the lower bound running time complexity of an algorithm. So if we represent a complexity of an algorithm in Omega notation, it means that the algorithm cannot be completed in less time than this, it would at-least take the time represented by Omega notation or it can take more (when not in best case scenario).



Theta Notation (Θ)

This notation describes both upper bound and lower bound of an algorithm so we can say that it defines exact asymptotic behavior. In the real case scenario, the algorithm not always run on best and worst cases, the average running time lies between best and worst and can be represented by the theta notation.





D. Mathematical Notations and Functions

Mathematics has been defined as the science of structure, order, and relation that has evolved from elemental practices of counting, measuring, and describing the shapes of objects. It deals with logical reasoning and quantitative calculation, and its development has involved an increasing degree of idealization and abstraction of its subject matter.

Mathematics form the algorithms of how program works. There are some mathematical functions that are being used as control structures and some to structure and manage your data. All such applications require using complex calculations/equations that are tedious to perform manually. Programmatically, such calculations would involve usage of logarithms, trigonometry, exponential equations, etc.

Now, you cannot have all the log or trigonometry tables hard-coded somewhere in your application or data. The data would be enormous and complex to maintain. Java provides a very useful class for this purpose. It is the Math java class (java.lang.Math). This class provides methods for performing the operations like exponential, logarithm, roots and trigonometric equations too.

Method	Description	Arguments
abs	Returns the absolute value of the argument	Double, float, int, long
round	Returns the closed int or long (as per the argument)	double or float
ceil	Returns the smallest integer that is greater than or equal to the argument	Double
floor	Returns the largest integer that is less than or equal to the argument	Double
min	Returns the smallest of the two arguments	Double, float, int, long
max	Returns the largest of the two arguments	Double, float, int, long

Absolute Value

In mathematics, the absolute value or modulus of a real number x , denoted $|x|$, is the non-negative value of x without regard to its sign. Namely, $|x| = x$ if x is positive, and $|x| = -x$ if x is negative, and $|0| = 0$. For example, the absolute value of 3 is 3, and the absolute value of -3 is also 3.

No Negatives! So in practice "absolute value" means to remove any negative sign in front of a number, and to think of all numbers as positive (or zero).

Get the absolute value of the following:

$|3.14| = \underline{\hspace{2cm}}$ $|-12.29| = \underline{\hspace{2cm}}$ $|-0.0001| = \underline{\hspace{2cm}}$ $|-9.81| = \underline{\hspace{2cm}}$



Rounding Off Numbers

Rounding means making a number simpler but keeping its value close to what it was.

- If the number you are rounding is followed by 5, 6, 7, 8, or 9, round the number up.
Example: 38 rounded to the nearest ten is 40.
- If the number you are rounding is followed by 0, 1, 2, 3, or 4, round the number down.
Example: 33 rounded to the nearest ten is 30.
- If the first figure dropped is 5, and all the figures following the five are zero or if there are no figures after the 5, then the last figure kept should be increased by 1 if that last figure is odd. For example, if only two decimals are to be kept, then 6.755000 becomes 6.76.

Round off the following numbers of the underlined digit:

77, <u>4</u> 53 = _____	59,8 <u>9</u> 9 = _____	1 <u>9</u> ,113 = _____	95, <u>1</u> 50 = _____
3.14 <u>1</u> 6 = _____	<u>0</u> .0005 = _____	14 <u>3</u> .44 = _____	999. <u>9</u> 5 = _____

Programming Equivalent

Math.abs

```
public class Guru99 {  
    public static void main(String args[]) {  
  
        int i1 = 27;  
        int i2 = -45;  
        double d1 = 84.6;  
        double d2 = 0.45;  
        System.out.println("Absolute value of i1: " + Math.abs(i1));  
  
        System.out.println("Absolute value of i2: " + Math.abs(i2));  
  
        System.out.println("Absolute value of d1: " + Math.abs(d1));  
  
        System.out.println("Absolute value of d2: " + Math.abs(d2));  
  
    }  
}
```

Output:

```
Absolute value of i1: 27  
Absolute value of i2: 45  
Absolute value of d1: 84.6  
Absolute value of d2: 0.45
```



Math.round

```
public class Guru99 {  
    public static void main(String args[]) {  
        double d1 = 84.6;  
        double d2 = 0.45;  
        System.out.println("Round off for d1: " + Math.round(d1));  
  
        System.out.println("Round off for d2: " + Math.round(d2));  
    }  
}
```

Output:

```
Round off for d1: 85  
Round off for d2: 0
```

Floor and Ceiling Functions

Ceil (short for ceiling) and floor function are both mathematical functions. It is often used in mathematical equations as well as in computer science in the likes of computer applications like spreadsheets, database programs, and computer languages like Java, C, C++, and Python.

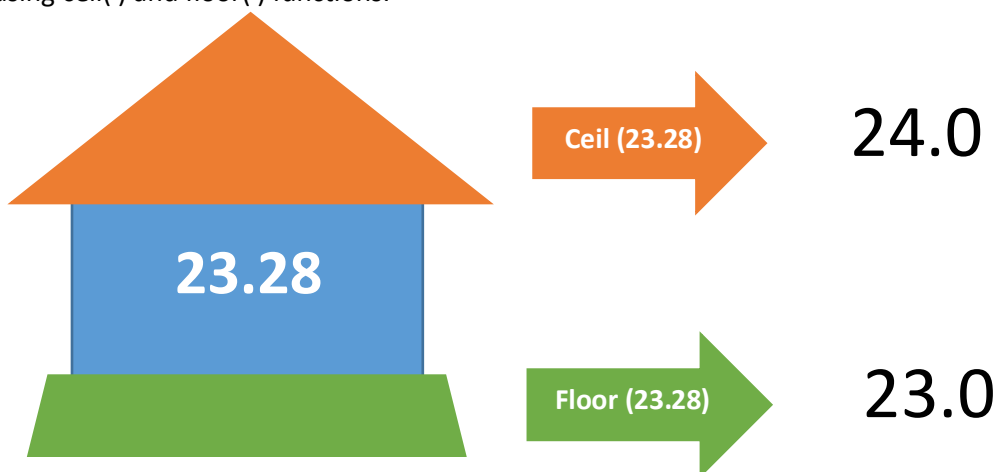
Ceil and floor functions are different in many respects. For example, ceil function returns the least value of the integer that is greater than or equal to the specified number. On the other hand, floor function gets the greatest value that is less than or equal to the specified number. The specified number is always a double precision number.

To better understand the Ceil and Floor functions, you can think of a house as illustrated below,

Ceil function uses this symbol $\lceil \quad \rceil$ that is similar to a roof, while

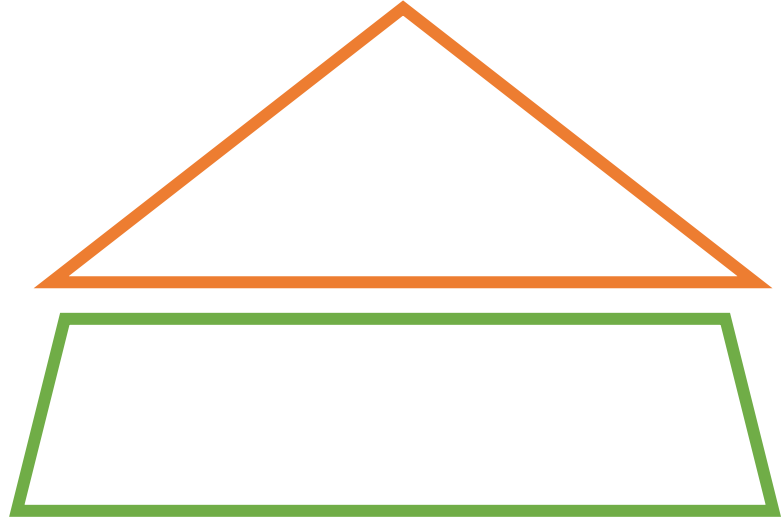
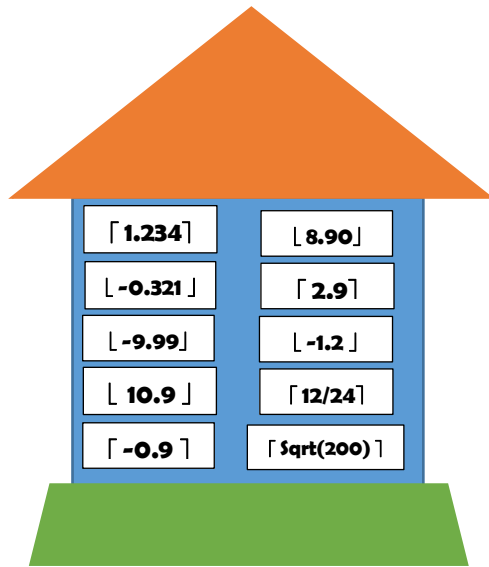
Floor function uses this symbol $\lfloor \quad \rfloor$ that is similar to a floor

A double data type was given floating in the house, and simplify it to the nearest integer value using ceil() and floor() functions.





Determine the equivalent value from each box and write your corresponding answer to the space provided in the diagram.



Min and Max Functions

The `java.lang.Math.min(int a, int b)` returns the smaller of two int values. That is, the result is the value closer to negative infinity. Example `Math.min (23, 17)` the result is 17.

`Math.max(int a, int b)` returns the greater of two int values. That is, the result is the argument closer to positive infinity. If the arguments have the same value, the result is that same value. Example `Math.max (-12, -29)` the result is -12.

How do you find the min and max of three given integers? Design an algorithm that determines the ***min*** and ***max*** between -35, -1, 15.



Programming Equivalent

Math.ceil & Math.floor

```
public class Guru99 {  
    public static void main(String args[]) {  
        double d1 = 84.6;  
        double d2 = 0.45;  
        System.out.println("Ceiling of '" + d1 + "' = " + Math.ceil(d1));  
  
        System.out.println("Floor of '" + d1 + "' = " + Math.floor(d1));  
  
        System.out.println("Ceiling of '" + d2 + "' = " + Math.ceil(d2));  
  
        System.out.println("Floor of '" + d2 + "' = " + Math.floor(d2));  
    }  
}
```

Output:

```
Ceiling of '84.6' = 85.0  
Floor of '84.6' = 84.0  
Ceiling of '0.45' = 1.0  
Floor of '0.45' = 0.0
```

Math.min

```
public class Guru99 {  
    public static void main(String args[]) {  
        int i1 = 27;  
        int i2 = -45;  
        double d1 = 84.6;  
        double d2 = 0.45;  
        System.out.println("Minimum out of '" + i1 + "' and '" + i2 + "' = " + Math.min(i1, i2));  
  
        System.out.println("Maximum out of '" + i1 + "' and '" + i2 + "' = " + Math.max(i1, i2));  
  
        System.out.println("Minimum out of '" + d1 + "' and '" + d2 + "' = " + Math.min(d1, d2));  
  
        System.out.println("Maximum out of '" + d1 + "' and '" + d2 + "' = " + Math.max(d1, d2));  
    }  
}
```

Output:

```
Minimum out of '27' and '-45' = -45  
Maximum out of '27' and '-45' = 27  
Minimum out of '84.6' and '0.45' = 0.45  
Maximum out of '84.6' and '0.45' = 84.6
```



Modulo Division or Remainder Function

The modulo operator is an arithmetic operator that is used to divide one operand by another and return the remainder as its result. Java has one important arithmetical operator you may not be familiar with, %, also known as the modulus or remainder operator. The % operator returns the remainder of two numbers. For instance, $10 \% 3$ is 1 because 10 divided by 3 leaves a remainder of 1.

The modulus operator returns the remainder of a division of one number by another. In most programming languages, modulo is indicated with a percent sign. For example, "4 mod 2" or "4%2" returns 0, because 2 divides into 4 perfectly, without a remainder. "5%2", however, returns 1 because 1 is the remainder of 5 divided by 2 (2 divides into 5 2 times, with 1 left over).

How will you test if the given number is an odd or even number using modulo division?

sqrt () and cbrt ()

The sqrt(double s) method of Math class can be used to find the square root of the double or int value. cbrt(double c) method of Math class can be used to find the cube root of the double or int value. Both the methods are static, so it can be called directly using class name without creating object.

Perform the following operations without using a calculator:

sqrt (81) = _____	sqrt (120) = _____	sqrt (500) = _____	sqrt (45.25) = _____
cbrt (125) = _____	cbrt (96) = _____	cbrt (1000) = _____	cbrt (15.9) = _____
$25^{1/2} =$ _____	$64^{1/3} =$ _____	$16^{1/4} =$ _____	$\frac{1}{(81)^{-1/2}} =$ _____



IV. TEACHING AND LEARNING MATERIALS RESOURCES

- PC Computer || Laptop || Android Phone
- Installed C/Java programming language
 - For downloads
 - C/C++ → <https://developerinsider.co/download-turbo-c-for-windows-7-8-8-1-and-windows-10-32-64-bit-full-screen/>
 - Java → <https://www.filehorse.com/download-java-runtime-64/download/>
 - For online compiler
 - C/C++/Java → https://www.onlinegdb.com/online_c_compiler
 - Java → <https://www.jdoodle.com/online-java-compiler/>
 - For mobile user, download in the App store
 - Java Editor
 - AIDE-IDE for Android Java C++

V. LEARNING TASKS

A. ENGAGE

Activity 1: Math is Amazing!

A.1. What is Pythagorean Theorem?

A.2. Design an algorithm that will look for the unknown side, given the two sides of a right triangle.

A.3. Implement your algorithm by designing a program in C or Java.

B.1. What is Quadratic Equation and when do you use it?



B.2. Design an algorithm that will solve for unknown values using quadratic equations.

B.3. Implement your algorithm by designing a program in C or Java.

B.4. Test your program by solving the following:

a. $-15x^2 + 11x - 2 = 0$ where $a = \square$, $b = \square$, and $c = \square$ b. $-4x^2 - 27x + 7 = 0$ where $a = \square$, $b = \square$, and $c = \square$

$$x = \frac{-(\) \pm \sqrt{(\)^2 - 4(\)(\)}}{2(\)}$$

$$x = \frac{-(\) \pm \sqrt{(\)^2 - 4(\)(\)}}{2(\)}$$

c. $8x^2 - 10x - 3 = 0$ where $a = \square$, $b = \square$, and $c = \square$ d. $2x^2 + 5x - 3 = 0$ where $a = \square$, $b = \square$, and $c = \square$

$$x = \frac{-(\) \pm \sqrt{(\)^2 - 4(\)(\)}}{2(\)}$$

$$x = \frac{-(\) \pm \sqrt{(\)^2 - 4(\)(\)}}{2(\)}$$

Rubric:

Completed the activities and understood the topic based on the given answer	Outstanding 50 points	Very Good 40 points	Good 30 points	Fair 20 points	No Work Output
---	--------------------------	------------------------	-------------------	-------------------	-------------------

B. EXPLORE & EXPLAIN

Design a program that you can use to solve the following:

1 a. $\frac{\sqrt{\frac{4355}{5}}}{\sqrt{975}}$	1 b. $\frac{\sqrt{243 + 245}}{\sqrt{698}}$
2 a. $\frac{\sqrt{2 \times 146}}{\sqrt{869}}$	2 b. $(\sqrt{\frac{1150}{2}})^2$



3 a. $\sqrt{374} \times \sqrt{2 \times 241}$	3 b. $\sqrt{\frac{2835}{5}} \times \sqrt{523}$
4 a. $\frac{\sqrt{1225 - 755}}{\sqrt{612}}$	4 b. $\sqrt{466} \times \sqrt{303 + 606}$
5 a. $\sqrt{\frac{1620}{4}} - \sqrt{1000}$	5 b. $\frac{\sqrt{4 \times 70}}{\sqrt{377}}$

Rubric:

Each correct answer will be given 3 points. Total score = 30 points	Question 1a	Question 2a	Question 3a	Question 4a	Question 5a	Total Score
	Question 1b	Question 2b	Question 3b	Question 4b	Question 5b	

C. ELABORATE & EVALUATION

I. Answer the following question by encircling the letter of your choice or submit your answer on a separate sheet of paper. (20 points)

- Two main measures for the efficiency of an algorithm are
 - Processor and memory
 - Complexity and capacity
 - Time and space
 - Data and space
- The time factor when determining the efficiency of algorithm is measured by
 - Counting microseconds
 - Counting the number of key operations
 - Counting the number of statements
 - Counting the kilobytes of algorithm
- The space factor when determining the efficiency of algorithm is measured by
 - Counting the maximum memory needed by the algorithm
 - Counting the minimum memory needed by the algorithm
 - Counting the average memory needed by the algorithm



- d. Counting the maximum disk space needed by the algorithm
4. Which of the following case does not exist in complexity theory
a. Best case b. Worst case c. Average case d. Null case
5. The complexity of the average case of an algorithm is
a. Much more complicated to analyze than that of worst case
b. Much more simpler to analyze than that of worst case
c. Sometimes more complicated and some other times simpler than that of worst case
d. None of the above
6. Big-Omega notation sets
a. a precise time complexity; b. a lower bound;
c. an upper bound; d. an upper and lower bound;
e. an indefinite estimate
7. For analyzing an algorithm, which has better computing time?
a. $O(100 \log N)$ b. $O(N \log N)$ c. $O(N)$ d. $O(N^2)$ e. $O(2N)$
8. For a problem of size n , a solution of the worst time complexity would be
a. 1 (constant time) b. 2 to the n power (exponential) c. n (linear)
d. none is worst e. n squared (quadratic);
9. $\log_2(100)$ is closest to
a. 1; b. 2; c. 6; d. 20; e. 100
10. $\log_2(1000)$ is closest to
a. 1; b. 2; c. 5; d. 10; e. 100

VI. REFERENCES

- Fox, C. (2018). Java Data Structures and Algorithms 1st Ed.
- Miller B. & Ranum D. (2013). Problem Solving with Algorithms and Data Structures Release 3.0
- Goodrich, M. & Tamassia, R. Data Structures and Algorithms in Java 4th Ed.
- Wikipedia
- Arya Ashish (2010). 10 Effective Ways to Become a Good Programmer
<https://dzone.com/articles/10-effective-ways-become-good>
- <http://what-when-how.com/compiler-writing/algorithmic-notation-compiler-writing-part-1/>
- <https://bisma.in/algorithm-and-its-characteristics/>
- <https://beginnersbook.com/2018/10/ds-asymptotic-notation/#:~:text=Asymptotic%20Notations%20are%20the%20expressions,the%20complexity%20of%20an%20algorithm.&text=Average%20Case%3A%20In%20which%20we,between%20best%20and%20worst%20case.>